

SDSC2005

Intro to Computational Social Science

FINAL PROJECT

Exploring the Interplay of Genetics and Lifestyle in Obesity

Course Instructor: Prof. ZHU Jonathan

Group 9:

KOJONGIAN Gracelynn (57887115)

KURNIAWAN Erica Cleodora (57889286)

YOLANDA Valeryn (57884567)

Semester B (2024-2025)

Abstract

This study explored the relative influence of lifestyle factors and genetic predisposition to obesity risk and their interplay across demographic groups. Logistic regression and p-value analysis were utilized on datasets from UC Irvine Machine Learning Repository. The research assessed the impact of various predictors on obesity likelihood. Key findings revealed that the model, considering only lifestyle factors obtained 70% accuracy on predicting obesity likelihood, but combining both, genetic factors included, showed 83% predictive accuracy. This suggests that genetic factors play a relatively minor role compared to lifestyle factors. Demographic analysis highlighted age-specific patterns: lifestyle factors (e.g., diet, screen time) dominated obesity risk in young adults, while metabolic markers (e.g., insulin) were pivotal for older adults. This research contributes to targeted public health strategies by highlighting the importance of lifestyle modifications in obesity prevention, while also considering genetic and demographic variations.

I. Introduction

In a nutshell, the project investigates the relative contribution of lifestyle factors and genetic predisposition on obesity risk and examines how their interplay varies across demographic subgroups. Then we also compare lifestyle factors and genetic predisposition, which one influenced obesity risk more. And of course also investigate how lifestyle and non lifestyle factors vary across demographic subgroups. The motivation for this research stems from the urgent need to inform targeted public health strategies, especially in light of the rising obesity rates linked to the prevalence of fast food and high-sugar diets worldwide.

The intended audience for this research includes both academic researchers and the general public. This study aims to raise awareness among the general public about the growing concerns surrounding obesity, which has become increasingly pressing each year. Furthermore, it seeks to provide academic researchers with a strengthened foundation for their studies on the factors contributing to obesity, utilizing a larger sample set for more robust findings. Therefore, the research purpose is explanatory.

II. Background

Obesity is a topic that has been studied many times by researchers with different focuses in each research. Based on our data, the data used were from NHANES, where the objective of the data was to predict age based on variables such as Age, Gender, BMI, diabetic type, blood glucose, and insulin level. At the same time, the second was regarding 'Estimation of Obesity Levels Based On Eating Habits and Physical Condition', with variables such as gender, age, family history, and lifestyle factors.

The paper was decided to be an explanatory and descriptive type of research because the two datasets used were mainly to predict whether a certain person with specific characteristics might potentially have obesity or not and to determine the predicted age based on the non-life-style factor. As a Computational Social Science course, the main idea is understanding the causal effects. Hence, the purpose of our study is to understand the cause and effect of obesity, but also to utilize machine learning algorithms for other purposes.

The reason the Obesity topic was chosen was based on the report given by the World Health Organization, which has released a report indicating that adult obesity has more than doubled since 1990, while adolescent obesity has quadrupled. This context informs the purpose of our research.

Based on research conducted by Fruh (2017) on the topic of obesity, the study examines the impact of obesity on health and well-being through various associated comorbidities, such as diabetes, cardiovascular disease, and cancer. Additionally, the researcher explores effective weight management strategies after the study. So, instead of exploring the broad factors contributing to obesity in general (age as a control variable), it is also a topic that has been extensively studied. We have chosen to separate the data by age group, focusing on young adults and adults. This approach aims to identify whether lifestyle or non-lifestyle factors are most influential in contributing to obesity using statistical methods. Furthermore, McPherson's (2007) study, "Genetic Contributors to Obesity," delves deeply into the molecular connection between genes and obesity. Her paper's conclusion suggested a positive genetic association with obesity, but it did not elaborate on the significance of this association. While our paper also discusses how genetics and obesity are intertwined, we aim to focus on the significance of genes and obesity. Additionally, the methodologies used in our study differ from those in McPherson's research. McPherson did not utilize machine learning models to determine whether genes significantly influence the likelihood of obesity in an individual. The discussion in this paper is based on the accuracy and precision of two models: one that incorporates both lifestyle and genetic factors, and another that focuses solely on lifestyle.

Therefore, this research focuses on answering the questions below:

1. What is the relative contribution of lifestyle factors versus genetic predisposition to obesity risk, and how does their interplay affect risk?
2. How do lifestyle factors differentially impact the risk of obesity compared to non-lifestyle factors?
3. How do lifestyle and non-lifestyle factors vary across demographic subgroups?

III. Method

A) Data

• Dataset

Both datasets are from the UCI Machine Learning Repository. The first dataset is about Estimation of Obesity Levels Based On Eating Habits and Physical Condition. This dataset contains cross-sectional, individual-level data from 2,111 participants, assessing obesity through self-reported eating habits and physical activity (Lifestyle Factors). Key variables include dietary patterns (e.g., high-calorie food intake, snacking frequency), exercise levels, and lifestyle factors (e.g., alcohol consumption, screen time).

The second dataset is about the National Health and Nutrition Health Survey. As a nationally representative survey of the U.S. population, NHANES collects robust individual and aggregate-level data, with the subset comprising information from 6,287 participants. The longitudinal design spans one year, enabling temporal analysis of health trends. While the dataset's primary focus is age prediction, it incorporates numerous obesity-related variables across multiple domains:

- Clinical measurements: Including body mass index (BMI)
- Biological markers: Such as diabetic type, insulin level, and blood glucose
- Demographic factors

• Data Preprocessing

Two datasets were used in understanding and achieving the purpose of this research. A major preprocessing challenge involves handling categorical data stored as object/string types (e.g., ordinal scales like "Never/Sometimes/Often"). Proper encoding (ordinal or one-hot) and type conversion are essential before model training. Additionally, potential biases from self-reported responses must be considered. Despite these preprocessing needs, the dataset provides valuable behavioral insights for obesity prediction, making it suitable for classification tasks (e.g., predicting obesity categories). Its focus on modifiable lifestyle factors allows for actionable public health applications, though its unspecified study duration limits temporal analysis. Additionally, the dataset was also split into 2, which are young adults (≤ 21 years old) and Adults (> 21 years old).

B) Metrics

Variables are categorized into dependent, independent, and control variables. The dependent variable is the obesity status, which is binary, distinguishing between obese and non-obese participants. The independent variables include various lifestyle factors, such as dietary patterns (e.g., high-calorie food intake, snacking frequency), exercise levels, alcohol consumption, and screen time. Additionally, clinical measurements, like Body Mass Index (BMI), glucose level, diabetic type, and insulin level, are

crucial indicators of health related to obesity. Biological markers, including cholesterol and blood glucose levels, further enhance the analysis.

The control variable in this study is gender, which is essential for accounting for potential differences in obesity risk and behaviors between male and female participants. A significant mediator in this research is diet, especially the maintenance of caloric intake. A balanced diet can reduce the risk of obesity, while physical activities also contribute to this effect.

The moderator variable identifies how different factors influence the relationship between lifestyle and obesity risk. Here, age serves as a moderator, indicating that adults may respond differently to clinical measurements or physical activity compared to younger adults. This understanding enables targeted public health interventions.

C) Machine Model and Statistics Method

- **Logistic Regression**

To systematically evaluate the determinants of obesity, this study used logistic regression as the primary analytical framework. As a supervised learning algorithm specifically designed for binary classification problems (obesity and non-obesity), logistic regression offered an optimal approach to assess how various predictors influence obesity likelihood through the accuracy of the model.

- **P-value**

This study used thorough statistical methods to evaluate the impact of lifestyle and non-lifestyle factors on obesity risk in various population groups. By analyzing p-values from logistic regression models to pinpoint the key predictors of obesity. In the analysis of the general population, all predictor variables were assessed to identify those that had statistically significant associations with obesity outcomes ($p < 0.05$). Furthermore, the overall distribution of p-values was also observed to see if lifestyle factors had a more significant role in obesity. Lastly, in regard to the exploration of these relationships within specific age groups, particularly adults over 21. This age-focused approach provided valuable insights into how the importance of different risk factors changes throughout life.

IV. Results

A) What is the relative contribution of lifestyle factors versus genetic predisposition to obesity risk, and how does their interplay affect risk?

To address the first question about the relative contributions of lifestyle factors versus genetic predisposition to obesity risk, a machine learning model was employed, specifically logistic regression.

Two logistic regression analyses were performed, one with lifestyle factors as the only independent variable and one with genetic factors as the independent variable. The precision with which these algorithms identified obese people was revealed by their accuracy.

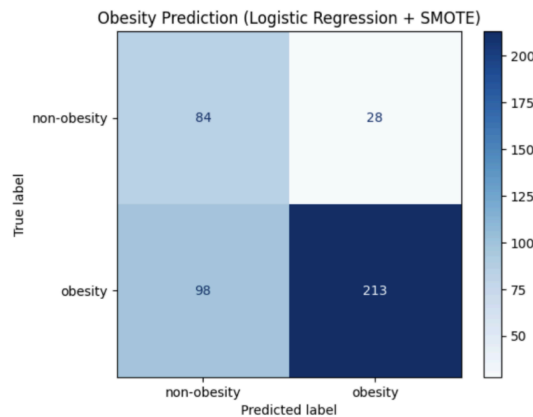


Figure 1. Obesity Model using Lifestyle
(Accuracy: 0.7)



Figure 2. Obesity Model using Lifestyle and Genetics
(Accuracy: 0.83)

Findings showed that the model considering only lifestyle factors achieved an accuracy of about 70%, while the model incorporating both lifestyle and genetic factors showed a higher accuracy of approximately 83%. This suggests that genetic factors play a relatively minor role in an individual's likelihood of gaining weight and becoming obese.

According to Harvard Health Publishing (2019), despite around 400 genes being linked to obesity, only a few are considered major contributors. These genes can influence obesity in various ways, including effects on appetite, satiety, metabolism, food cravings, and body fat distribution. Furthermore, Cardiol (2007) estimates that the heritability of obesity is around 40%.

Obesity risk significantly increases when poor lifestyle decisions are combined with hereditary factors. The main cause of this complicated problem is the body storing too many calories as fat. High-calorie foods, heavy in fat and sugar, are usually the source of extra calories, which the body stores as fat when physical activity is minimal (World Health Organization, n.d.).

This is why lifestyle factors are critical in our model. For instance, in the model it includes variables such as alcohol consumption, which is high in sugar. Additionally, managing calorie intake is essential, as the recommended daily caloric intake differs slightly between genders—approximately 2,500 kcal for men and 2,000 kcal for women (Zelman, 2024).

Moreover, many individuals fail to meet the recommended levels of physical activity, leading to further calorie surplus that the body stores as fat (Leitzmann, 2017). Thus, while genetics may set a baseline risk for obesity, lifestyle choices are crucial in how that risk manifests. A high genetic predisposition requires proactive changes to counteract this risk, whereas a low genetic risk simply raises the threshold at which unhealthy habits can lead to weight gain.

B) How do lifestyle factors differentially impact the risk of obesity compared to non-lifestyle factors?

LIFESTYLE FACTORS	
Feature	P-Value
no_meals	0
snacking	0
screen_time	2.22e-16
alcohol	6.83e-11
water_intake	5.97e-09
physical_activity	3.58e-08
high_calorie_intake	1.30e-06
calorie_monitoring	2.10e-02

Table 1. Statistical Significance of Lifestyle Factors

NON LIFESTYLE FACTORS	
Feature	P-Value
insulin_level	3.84e-77
intense_sport	1.02e-02
diabetic_type	1.25e-02
oral	5.55e-02
blood_glucose	3.07e-01

Table 2. Statistical Significance of Non-Lifestyle Factors

From the analysis, it is shown that lifestyle factors are more statistically significant to obesity rather than non-lifestyle factors. This can be seen by the consistency of all features from the lifestyle factors having p-value below 0.05, while for non-lifestyle factors, there are 2 features whose p-value is above 0.05. According to Investopedia, a p-value less than 0.05 is typically considered to be statistically significant.

Lifestyle factors such as meal frequency, snacking habits, phone screen time, alcohol consumption, water intake, physical activity, high calorie intake, and calorie monitoring, all have crucial roles in influencing weight gain and obesity risk. According to Paoli et al. (2019), the number of meals and snacks have a positive relationship with weight gain. Higher eating frequency combined with snacking habits can increase food stimuli, hunger, and desire to eat, which leads to higher calorie intake. When a human body gets more calories than it is needed, it will store the extra calorie as fat which will cause

weight gain (NHS, 2023). Alcohol consumption also contributes to excess energy intake as it adds extra calories to daily diet (Department of Health, 2023). Moreover, phone screen time is usually linked to physical activity, where those with higher screen time are believed to have lower physical activity. When physical activity levels are insufficient, the body stores excess energy as fat, causing a higher risk of obesity. As for water intake, drinking water increases the human body's energy expenditure, leading to more calories being burned and lower the risk of obesity (HR NewsWire, 2020.).

In contrast, out of all five features in non-lifestyle factors, only insulin level, sport intensity, and diabetic type shows significant association with obesity. Even though these non-lifestyle factors also have a correlation with obesity, they do not affect obesity the same way as lifestyle factors do. While lifestyle factors influence obesity by energy intake and expenditure, non-lifestyle factors affect obesity through metabolic and hormonal. Insulin level, sport intensity, and diabetic type, all affect how the human body releases and stores energy from food (Medical News Today, n.d.).

Furthermore, this result suggests that lifestyle factors are often modifiable and can be controlled by each individual, making them more impactful in obesity prevention. Lifestyle factors can lead to change in energy intake and expenditure, which play a more direct role in causing obesity rather than the non-lifestyle factors. Thus, effective obesity prevention strategies should focus on modifying lifestyle behaviors while also considering the biological context of individuals.

C) How do lifestyle and non-lifestyle factors vary across demographic subgroups?

YOUNG ADULTS

	Lifestyle	P-Value	Non Lifestyle	P-value
1	Transport	3.63 e ⁻¹⁰	Insulin Level	8.18 e ⁻²⁰
2	Snacking	2.00 e ⁻⁷	Oral	3.27 e ⁻²
3	Calorie Monitoring	4.43 e ⁻⁴	Blood Glucose	5.85 e ⁻²

Table 3. Top 3 Factors for Young Adults

ADULTS

	Lifestyle	P-Value	Non Lifestyle	P-value
1	High-Calorie Intake	3.63 e ⁻¹⁰	Insulin Level	1.27e ⁻⁶⁴
2	Snacking	2.00 e ⁻⁷	Intense Sport	4.26 e ⁻⁴
3	Water-Intake	4.43 e ⁻⁴	Blood-Glucose	1.20 e ⁻¹

Table 4. Top 3 Factors for Adults

P-values were used to identify which factors significantly contributed to obesity risk by analyzing a dataset of adults over 21 and young adults under 21. The findings indicate that for young adults, lifestyle factors—such as transportation habits and snacking—are more significant in predicting obesity risk. In contrast, non-lifestyle factors like insulin and glucose levels play a more prominent role for older adults. This distinction highlights how the influences on obesity risk can vary significantly across different age groups.

As individuals age, physiological changes can significantly alter metabolic processes, particularly those involving glucose regulation and insulin sensitivity. One of the key factors contributing to elevated blood glucose levels is insulin resistance, this condition tends to worsen with age. Insulin resistance results from cells in the body becoming less sensitive to insulin, which hinders their capacity to absorb glucose effectively. This dysfunction causes two significant metabolic disturbances: the liver produces more glucose and the muscles absorbing less glucose. Hyperglycemia (high blood sugar) results from elevated blood glucose levels. (Chia et al, 2018)

Obesity is a major contributor to the development of insulin resistance. Excess adipose tissue, particularly visceral fat, promotes chronic low-grade inflammation and the release of fatty acids, both of which interfere with insulin signaling pathways. As insulin resistance progresses, the pancreas compensates by secreting higher amounts of insulin, leading to hyperinsulinemia (elevated insulin levels in the blood). Paradoxically, this excess insulin exacerbates weight gain by promoting lipogenesis (the synthesis of new fat) while simultaneously inhibiting lipolysis (the breakdown of stored fat). Thus, a vicious cycle emerges: obesity worsens insulin resistance, insulin resistance leads to hyperinsulinemia, and hyperinsulinemia further contributes to fat accumulation and weight gain (Martin et al., 2014).

Young adults often encounter lifestyle challenges that contribute to obesity, including poor dietary choices and sedentary behavior. Research highlights several key findings regarding their eating habits. First, young adults value their independence and often prioritize making their own food choices, which can lead to less mindful eating. Second, social interactions frequently take precedence over nutritional considerations; many young adults choose food based on their social environment, such as opting for fast food when dining out with friends. Third, financial constraints play a significant role, as many young adults feel compelled to select unhealthy, inexpensive, and convenient food options (National Institute for Health and Care, n.d.).

These factors can lead to poor dietary choices, impacting daily calorie intake and increasing the risk of weight gain and obesity. Furthermore, the influence of screen time cannot be overlooked. Young adults engage with a wider range of technologies than older people, resulting in increased time spent on screens, whether via computers, smartphones, or television. This increase in screen time often correlates with sedentary lifestyles, reducing overall calorie expenditure and promoting weight gain (Olson et al., 2012). Addressing these lifestyle factors is crucial for mitigating obesity risk in young adults.

V. Conclusion

In conclusion, this research shows the significant impact of lifestyle factors on obesity risk compared to genetic predisposition. The result from the logistic regression shows that lifestyle factors can accurately determine obesity about 70%, but it is also undeniable that genetic factors have an impact. The model's accuracy increased to 83% when combined with genetic factors.

Lifestyle factors (e.g. meal frequency, snacking habits, phone screen time, alcohol consumption, water intake, physical activity, high calorie intake, and calorie monitoring) contributed directly to energy intake and expenditure, a primary factor of weight gain. While non-lifestyle factors affect obesity through metabolic and hormonal. This suggests that lifestyle factors are modifiable and obesity prevention should focus on that.

Demographics play an important role in the variation of these factors. It is found that the factors are different among distinct age-groups. For young adults, lifestyle choices significantly impact obesity risk, including poor dietary choices and sedentary behavior. For instance, many young adults prioritize their independence and food choices which can lead to less mindful dietary choices and increased calorie consumption. On the other hand, non-lifestyle factors are more strongly associated with older persons, especially metabolic problems like insulin resistance that exacerbate with age and fat.

This difference emphasises the need for customised solutions that take into account the particular difficulties encountered by certain demographic groups. Lifestyle factors are modifiable, even if a genetic predisposition creates the conditions for obesity risk. Focusing on lifestyle factors is crucial since the greatest strategy to reduce the risk of obesity is to change eating habits or eating behaviours (e.g., snacking less or more healthily), be physically active on a regular basis, and spend less time in sedentary behaviour.

The root cause of the research's limitation may be found due to the data itself since it is considered a self reported data, where there is a high chance that it may lead to biases and inaccuracy on the result. Additionally, there may be confounding variables that we did not touch on that may have an impact towards obesity such as psychological factors. Where conditions like depression can lead to lack of exercising impacting weight independently. Therefore, future research should explore these factors while also collaborating with experts in psychology, sociology, and nutrition for further understanding how confounding factors, such as socioeconomic status and psychological issues, influence obesity.

References

- Beers, B. (2024). *P-Value: What It Is, How to Calculate It, and Examples*. Investopedia.
<https://www.investopedia.com/terms/p/p-value.asp#:~:text=A%20p%2Dvalue%20less%20than,null%20hypothesis%20is%20not%20rejected.>
- Chia, C., & Egan, J. (2019). *Age-related changes in glucose metabolism, hyperglycemia, and cardiovascular risk*. PMC Home. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6205735/>
- Department of Health. (2023). Alcohol Consumption and Obesity Risk. Department of Health.
https://www.chp.gov.hk/files/pdf/ncd_watch_feb2023_eng.pdf
- Diabetes and metabolism: What to know*. (2021). MedicalNewsToday.
<https://www.medicalnewstoday.com/articles/diabetes-and-metabolism>
- Fischer, K. (2024, June 29). *Food choices: What influences young people's food choices?*. NIHR Evidence.
<https://evidence.nihr.ac.uk/alert/what-influences-young-peoples-food-choices/>
- Leitzmann, M. (2017). *Physical activity, sedentary behaviour, and obesity - Energy balance and obesity - NCBI bookshelf*. National Center for Biotechnology Information.
<https://www.ncbi.nlm.nih.gov/books/NBK565813/>
- Martyn, J. (2014). *Obesity-induced insulin resistance and hyperglycemia: Etiological factors and molecular mechanisms*. PMC Home. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3896971/>
- Olson, K., O'Brien, M., & Rogers, W. (2012). *Diffusion of technology: Frequency of use for younger and older adults*. PMC Home. <https://pmc.ncbi.nlm.nih.gov/articles/PMC3370300/>
- Paoli, A., Tinsley, G., Bianco, A., & Moro, T. (2019). The Influence of Meal Frequency and Timing on Health in Humans. *The Role of Fasting. Nutrients*. 11(4), 719.
<https://doi.org/10.3390/nu11040719>

Understanding calories. (2023). NHS.

<https://www.nhs.uk/live-well/healthy-weight/managing-your-weight/understanding-calories/#:~:text=For%20example%3A,fewer%20calories%20than%20you%20use.>

WHO. (2020). *Obesity*. World Health Organization (WHO).

https://www.who.int/health-topics/obesity#tab=tab_1

Why people become overweight. (2019). Harvard Health.

<https://www.health.harvard.edu/staying-healthy/why-people-become-overweight>

Yes, drinking more water may help you lose weight. (2020). HR NewsWire.

<https://hub.jhu.edu/at-work/2020/01/15/focus-on-wellness-drinking-more-water/#:~:text=Science%20suggests%20that%20water%20can,to%20results%20on%20the%20scale.>

Zelman, K. M., & LD. (2009). *How many calories do you really need?* WebMD.

<https://www.webmd.com/diet/calories-chart>

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import confusion_matrix
```

```
df = pd.read_csv('NHANES.csv')
```

```
df.dropna()
```

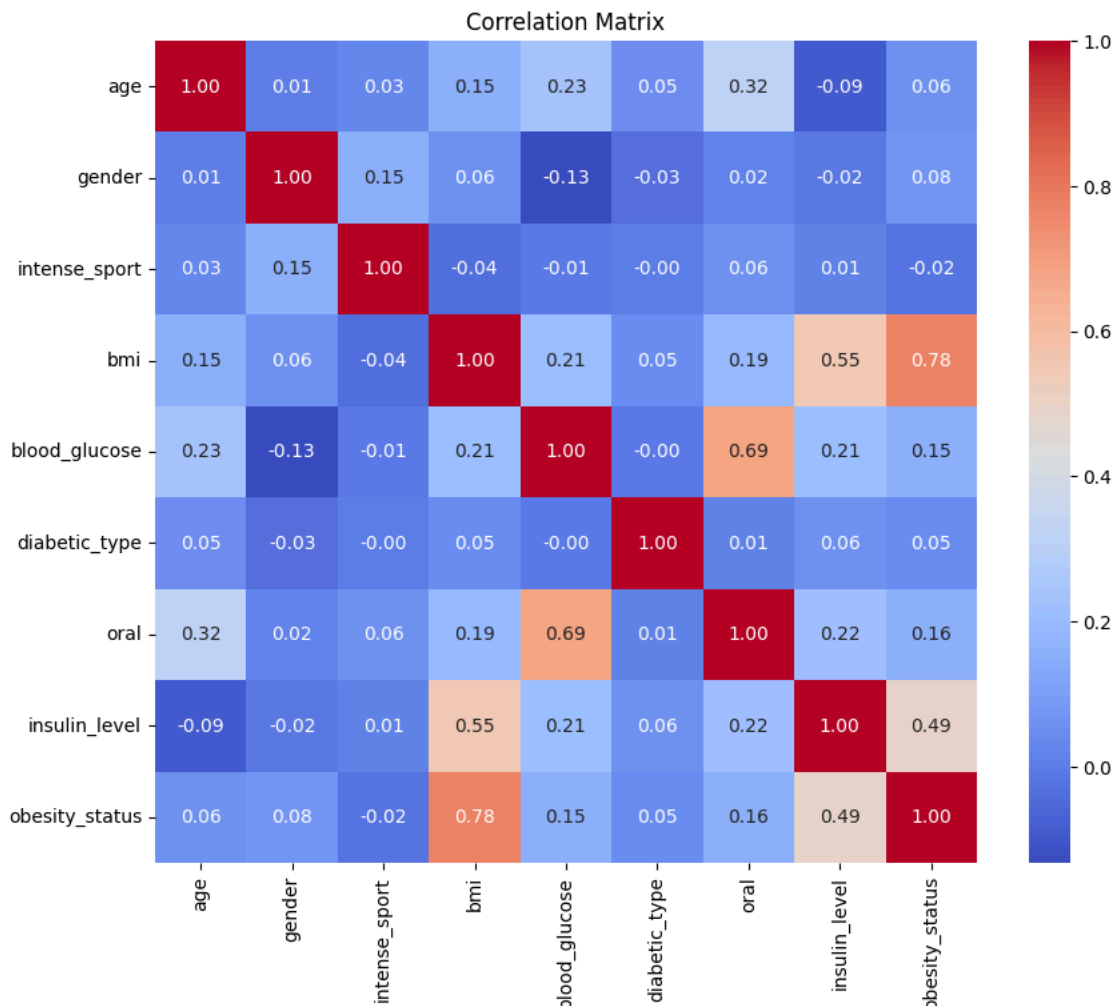


	seq_num	age_group	age	gender	intense_sport	bmi	blood_glucose	diabetic_type	oral	insulin_level
0	73564.0	Adult	61.0	2.0	2.0	35.7	110.0	2.0	150.0	14.91
1	73568.0	Adult	26.0	2.0	2.0	20.3	89.0	2.0	80.0	3.85
2	73576.0	Adult	16.0	1.0	2.0	23.2	89.0	2.0	68.0	6.14
3	73577.0	Adult	32.0	1.0	2.0	28.9	104.0	2.0	84.0	16.15
4	73580.0	Adult	38.0	2.0	1.0	35.9	103.0	2.0	81.0	10.92
...	...	...	...	...	...	...	...	...	...	...
2273	83711.0	Adult	38.0	2.0	2.0	33.5	100.0	2.0	73.0	6.53
2274	83712.0	Adult	61.0	1.0	2.0	30.0	93.0	2.0	208.0	13.02
2275	83713.0	Adult	34.0	1.0	2.0	23.7	103.0	2.0	124.0	21.41
2276	83718.0	Adult	60.0	2.0	2.0	27.4	90.0	2.0	108.0	4.99
2277	83727.0	Adult	26.0	1.0	2.0	24.5	108.0	2.0	108.0	3.76

2278 rows x 10 columns

```
df['obesity_status'] = df['bmi'].apply(lambda x: 1 if x >= 30 else 0)
```

```
filtered = df.drop(columns=['seq_num', 'age_group'])
cm = filtered.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Matrix')
plt.show()
```



```
df_a = df[df["age_group"] == "Adult"]
```

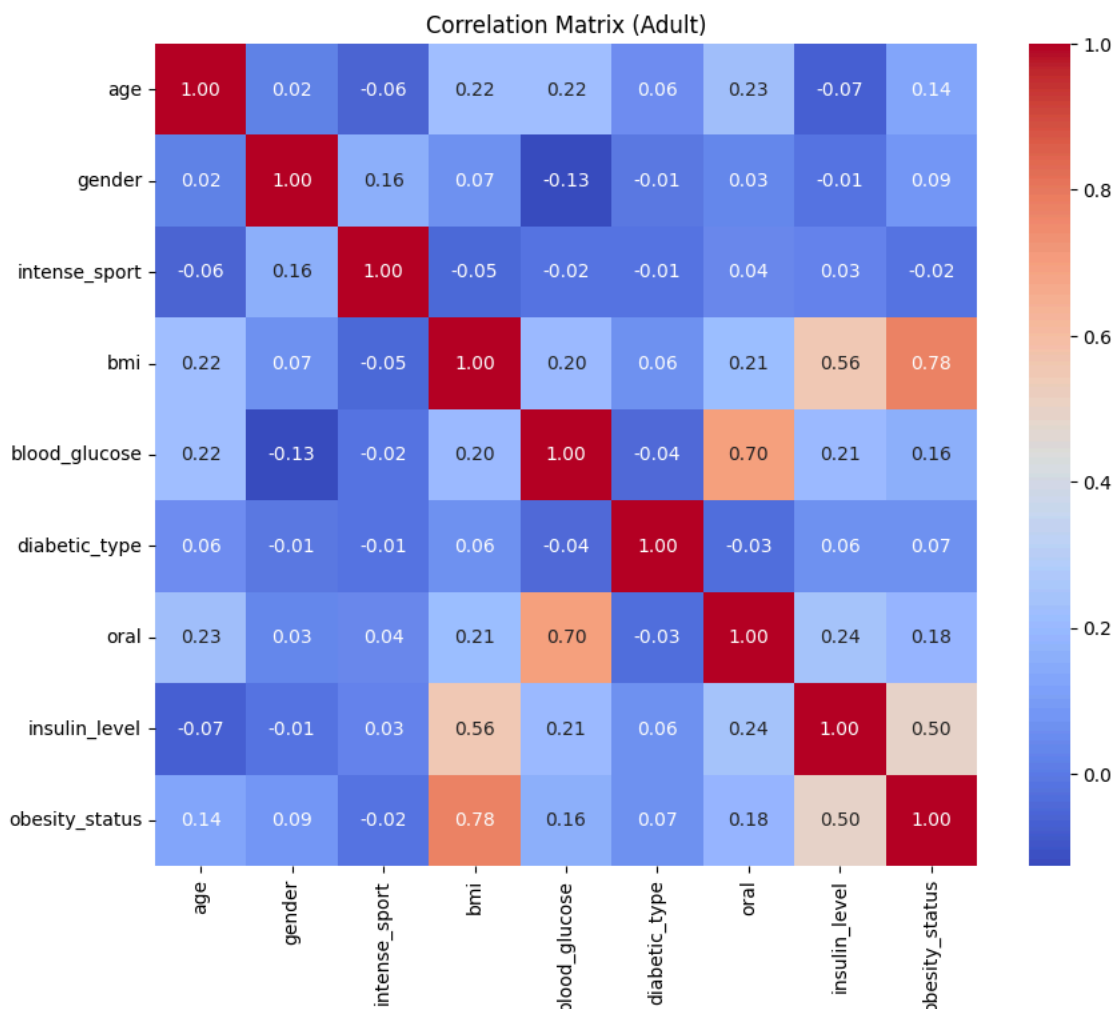
```
df_a['obesity_status'] = df_a['bmi'].apply(lambda x: 1 if x >= 30 else 0)
df_a.head()
```

<ipython-input-105-d282ca799021>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-slice-of-a-dataframe
df_a['obesity_status'] = df_a['bmi'].apply(lambda x: 1 if x >= 30 else 0)

	seq_num	age_group	age	gender	intense_sport	bmi	blood_glucose	diabetic_type	oral	insulin_level	obesity_status
0	73564.0	Adult	61.0	2.0	2.0	35.7	110.0	2.0	150.0	14.91	1
1	73568.0	Adult	26.0	2.0	2.0	20.3	89.0	2.0	80.0	3.85	0
2	73576.0	Adult	16.0	1.0	2.0	23.2	89.0	2.0	68.0	6.14	0
3	73577.0	Adult	32.0	1.0	2.0	28.9	104.0	2.0	84.0	16.15	0
4	73580.0	Adult	38.0	2.0	1.0	35.9	103.0	2.0	81.0	10.92	1

```
filtered_a = df_a.drop(columns=['seq_num', 'age_group'])
cma = filtered_a.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(cma, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Matrix (Adult)')
plt.show()
```



```
df_s = df[df["age_group"] == "Senior"]
```

```
df_s['obesity_status'] = df_s['bmi'].apply(lambda x: 1 if x >= 30 else 0)
df_s.head()
```

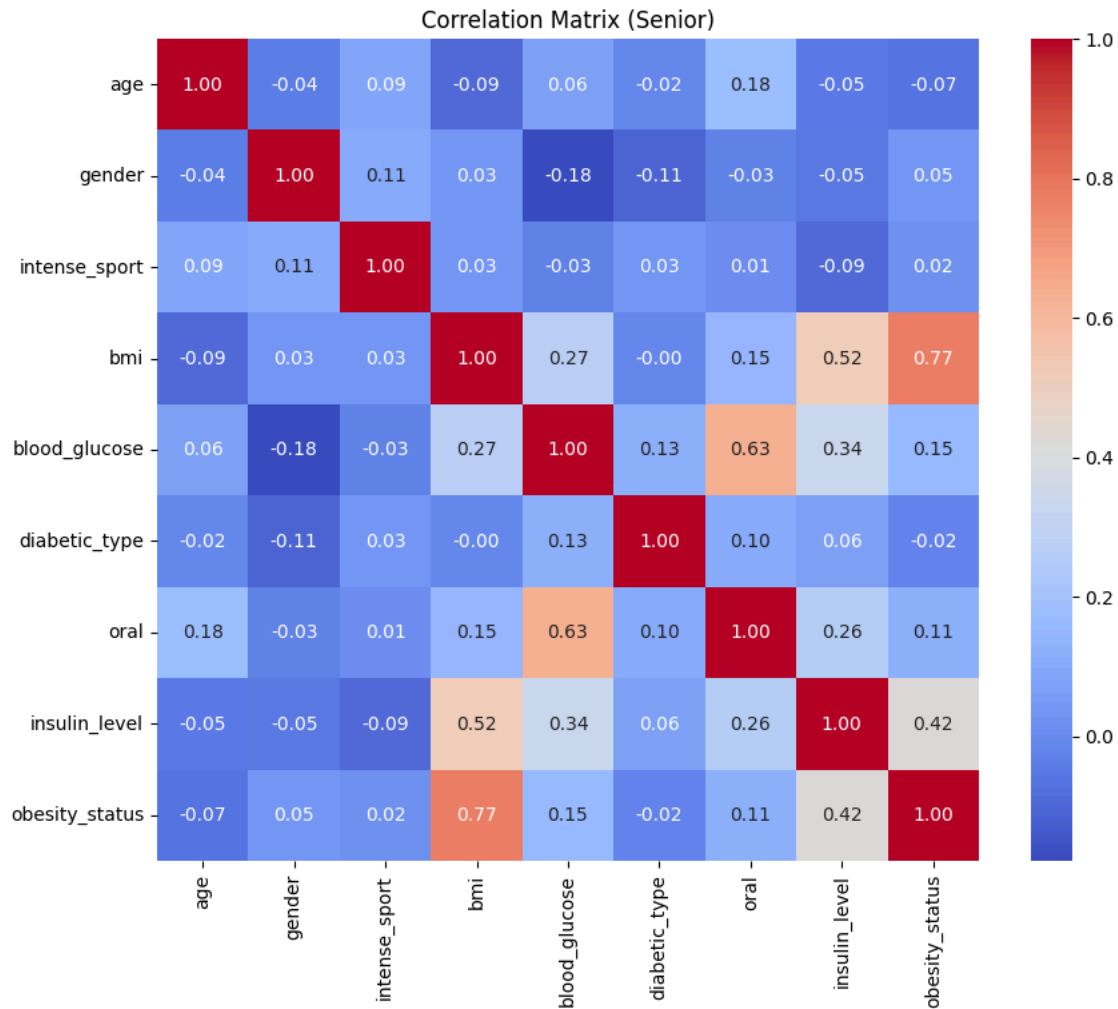


<ipython-input-107-413dc6bf2316>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-slice-of-a-dataframe
df_s['obesity_status'] = df_s['bmi'].apply(lambda x: 1 if x >= 30 else 0)

	seq_num	age_group	age	gender	intense_sport	bmi	blood_glucose	diabetic_type	oral	insulin_level	obesity_status
8	73607.0	Senior	75.0	1.0	2.0	38.9	89.0	2.0	113.0	17.47	1
12	73621.0	Senior	80.0	1.0	2.0	24.6	100.0	2.0	97.0	4.33	0
14	73639.0	Senior	71.0	1.0	2.0	30.3	133.0	2.0	295.0	22.92	1
15	73640.0	Senior	67.0	2.0	1.0	22.1	114.0	2.0	150.0	10.09	0
18	73659.0	Senior	70.0	2.0	2.0	46.1	139.0	2.0	154.0	42.67	1

```
filtered_s = df_s.drop(columns=['seq_num', 'age_group'])
cms = filtered_s.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(cms, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Matrix (Senior)')
plt.show()
```



```
X1 = df[['intense_sport', 'blood_glucose', 'diabetic_type', 'oral', 'insulin_level']]
y1 = df['obesity_status']
X_train1, X_test1, y_train1, y_test1 = train_test_split(X1, y1, test_size=0.2, random_state=42)
model1 = LogisticRegression()
model1.fit(X_train1, y_train1)
y_pred1 = model1.predict(X_test1)
```

```
accuracy1 = accuracy_score(y_test1, y_pred1)
precision1 = precision_score(y_test1, y_pred1)
recall1 = recall_score(y_test1, y_pred1)
f11 = f1_score(y_test1, y_pred1)
```

```
print(f"Accuracy: {accuracy1:.2f}")
print(f"Precision: {precision1:.2f}")
print(f"Recall: {recall1:.2f}")
print(f"F1-score: {f11:.2f}")
```

Accuracy: 0.76
Precision: 0.72
Recall: 0.37
F1-score: 0.49

```
# 1. Remove constant columns
constant_cols = [col for col in X1.columns if X1[col].nunique() == 1]
if constant_cols:
    X1 = X1.drop(columns=constant_cols)
```

```
# 2. Add intercept
X1_const = sm.add_constant(X1)
```

```
# 3. Fit model with robust solver
result = sm.Logit(y1, X1_const).fit(method='lbfgs', disp=0)
```

```
# 4. Prepare results - exclude 'const' and sort by p-value
feature_importance = pd.DataFrame({
    'Feature': result.params.index,
    'Coefficient': result.params.values,
    'P-Value': result.pvalues.values
})
```

```
# Remove constant row if exists
feature_importance = feature_importance[feature_importance['Feature'] != 'const']

# Sort by p-value
feature_importance = feature_importance.sort_values('P-Value')

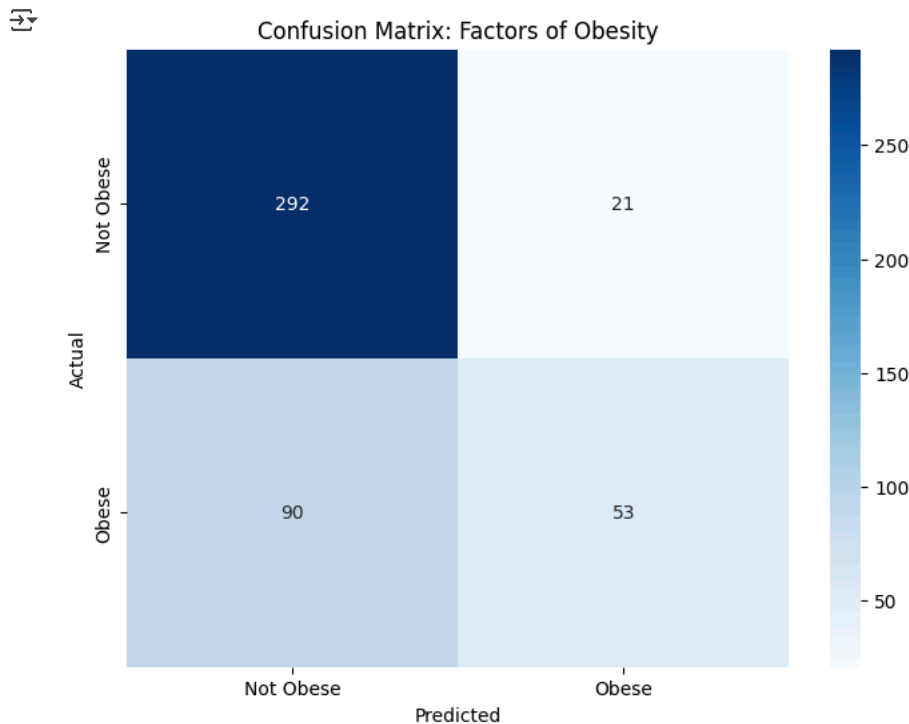
print("Feature Importance (sorted by P-value):")
print(feature_importance)
```

↗ Feature Importance (sorted by P-value):

	Feature	Coefficient	P-Value
5	insulin_level	0.163734	3.837646e-77
1	intense_sport	-0.346679	1.010740e-02
3	diabetic_type	-0.748525	1.248420e-02
4	oral	0.002969	5.557929e-02
2	blood_glucose	-0.004024	3.071765e-01

/usr/local/lib/python3.11/dist-packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered in exp
return 1/(1+np.exp(-X))
/usr/local/lib/python3.11/dist-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_

```
cm = confusion_matrix(y_test1, y_pred1)
plt.figure(figsize = (8, 6))
sns.heatmap(cm, annot = True, fmt = 'd', cmap = 'Blues', xticklabels = ['Not Obese', 'Obese'], yticklabels = ['Not Obese', 'Obese'],
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix: Factors of Obesity')
plt.show()
```



Adult Logistic Regression

```
X2 = df_a[['intense_sport', 'blood_glucose', 'diabetic_type', 'oral', 'insulin_level']]
y2 = df_a['obesity_status']
X_train2, X_test2, y_train2, y_test2 = train_test_split(X2, y2, test_size=0.2, random_state=42)
model2 = LogisticRegression()
model2.fit(X_train2, y_train2)
y_pred2 = model2.predict(X_test2)

accuracy2 = accuracy_score(y_test2, y_pred2)
precision2 = precision_score(y_test2, y_pred2)
recall2 = recall_score(y_test2, y_pred2)
f12 = f1_score(y_test2, y_pred2)

print(f"Accuracy: {accuracy2:.2f}")
print(f"Precision: {precision2:.2f}")
print(f"Recall: {recall2:.2f}")
print(f"F1-score: {f12:.2f}")
```

↗ Accuracy: 0.77
Precision: 0.66
Recall: 0.45

F1-score: 0.54

```
# 1. Remove constant columns
constant_cols2 = [col for col in X2.columns if X2[col].nunique() == 1]
if constant_cols2:
    X2 = X2.drop(columns=constant_cols2)

# 2. Add intercept
X2_const = sm.add_constant(X2)

# 3. Fit model with robust solver
result2 = sm.Logit(y2, X2_const).fit(method='lbfgs', disp=0)

# 4. Prepare results - exclude 'const' and sort by p-value
feature_importance2 = pd.DataFrame({
    'Feature': result2.params.index,
    'Coefficient': result2.params.values,
    'P-Value': result2.pvalues.values
})

# Remove constant row if exists
feature_importance2 = feature_importance2[feature_importance2['Feature'] != 'const']

# Sort by p-value
feature_importance2 = feature_importance2.sort_values('P-Value')

print("Feature Importance for Adult (sorted by P-value):")
print(feature_importance2)
```

```
↗ Feature Importance for Adult (sorted by P-value):
```

	Feature	Coefficient	P-Value
5	insulin_level	0.165557	9.295631e-67
1	intense_sport	-0.715626	3.751856e-07
4	oral	0.004923	7.701094e-03
2	blood_glucose	-0.007811	7.269973e-02
3	diabetic_type	-0.484404	1.861311e-01

```
/usr/local/lib/python3.11/dist-packages/statsmodels/discrete/discrete_model.py:2385: RuntimeWarning: overflow encountered in exp
return 1/(1+np.exp(-X))
/usr/local/lib/python3.11/dist-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge.
warnings.warn("Maximum Likelihood optimization failed to converge. Check your ML model specification.")
```

Senior Logistic Regression

```
X3 = df_s[['intense_sport', 'blood_glucose', 'diabetic_type', 'oral', 'insulin_level']]
y3 = df_s['obesity_status']
X_train3, X_test3, y_train3, y_test3 = train_test_split(X3, y3, test_size=0.2, random_state=42)
model3 = LogisticRegression()
model3.fit(X_train3, y_train3)
y_pred3 = model3.predict(X_test3)

accuracy3 = accuracy_score(y_test3, y_pred3)
precision3 = precision_score(y_test3, y_pred3)
recall3 = recall_score(y_test3, y_pred3)
f13 = f1_score(y_test3, y_pred3)

print(f"Accuracy: {accuracy2:.2f}")
print(f"Precision: {precision2:.2f}")
print(f"Recall: {recall2:.2f}")
print(f"F1-score: {f12:.2f}")
```

```
↗ Accuracy: 0.77
Precision: 0.66
Recall: 0.45
F1-score: 0.54
```

```
# 1. Remove constant columns
constant_cols3 = [col for col in X3.columns if X3[col].nunique() == 1]
if constant_cols3:
```

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
```

```
lifesytle = pd.read_csv("lifestyle.csv")
lifesytle.dtypes
```

**0**

Gender	object
Age	float64
Height	float64
Weight	float64
family_history_with_overweight	object
FAVC	object
FCVC	float64
NCP	float64
CAEC	object
SMOKE	object
CH2O	float64
SCC	object
FAF	float64
TUE	float64
CALC	object
MTRANS	object
NObeyesdad	object

dtype: object

```
lifesytle_cleaned = lifesytle.rename(columns={
    'FAVC': 'high_calorie_intake',
    'FCVC': 'Veggie_intake',
    'NCP' : 'no_meals',
    'CAEC' : 'snacking',
    'SMOKE' : 'smoker',
    'CH2O' : 'water_intake',
    'SCC' : 'calorie_monitoring',
    'FAF' : 'physical_activity',
```

```

    'TUE' : 'screen_time',
    'CALC' : 'alcohol',
    'MTRANS': 'transport',
    'NObesidad': 'obesity_level'
})

```

```
lifesytle_cleaned['family_history_with_overweight'] = lifesytle_cleaned['family_h
```

```
lifesytle_cleaned['Gender'] = lifesytle_cleaned['Gender'].map({'Male': 0, 'Female
```

```
lifesytle_cleaned['high_calorie_intake'] = lifesytle_cleaned['high_calorie_intake
```

```
lifesytle_cleaned['snacking'] = lifesytle_cleaned['snacking'].map({'Always': 3, '
```

```
# Step 1: Standardize values
```

```
lifesytle_cleaned['smoker'] = (
    lifesytle_cleaned['smoker']
    .astype(str)
    .str.strip()
    .str.lower()
)
```

```
# Step 2: Flexible mapping (catches common variants)
```

```
mapping = {
    'yes': 1, 'y': 1, '1': 1, 'true': 1,
    'no': 0, 'n': 0, '0': 0, 'false': 0
}
```

```
lifesytle_cleaned['smoker'] = (
    lifesytle_cleaned['smoker']
    .map(mapping)
    .fillna(-1) # Assign -1 to unmapped values (optional)
)
```

```
# Verify
```

```
print(lifesytle_cleaned['smoker'].value_counts())
```

```

⇒ smoker
0      2067
1        44
Name: count, dtype: int64

```

```
mapping = {
    'yes': 1, 'y': 1, '1': 1,
    'no': 0, 'n': 0, '0': 0
}
```

```
lifesytle_cleaned['calorie_monitoring'] = (
    lifesytle_cleaned['calorie_monitoring']
    .astype(str).str.lower().str.strip()
    .map(mapping)
)
```

```
.fillna(-1)
)
```

```
lifesytle_cleaned['alcohol'] = lifesytle_cleaned['alcohol'].map({
    'Always': 3,
    'Sometimes': 2,
    'Frequently': 1,
    'no': 0
})
```

```
distinct_values = lifesytle['CALC'].unique()
distinct_values
```

```
⇒ array(['no', 'Sometimes', 'Frequently', 'Always'], dtype=object)
```

```
lifesytle_cleaned['transport'] = lifesytle_cleaned['transport'].map({'Public_Tran
```

```
lifesytle_cleaned['obesity_level'] = lifesytle_cleaned['obesity_level'].map({
    'Insufficient_Weight': 0,
    'Normal_Weight': 0,
    'Overweight_Level_I': 1,
    'Overweight_Level_II': 1,
    'Obesity_Type_I': 1,
    'Obesity_Type_II': 1,
    'Obesity_Type_III': 1
})
```

```
lifesytle_cleaned
```

```
⇒
```

	Gender	Age	Height	Weight	family_history_with_overweight	high_blood_pressure
0	1	21.000000	1.620000	64.000000	1	1
1	1	21.000000	1.520000	56.000000	1	1
2	0	23.000000	1.800000	77.000000	1	1
3	0	27.000000	1.800000	87.000000	0	0
4	0	22.000000	1.780000	89.800000	0	0
...	...	...	...	...	...	...
2106	1	20.976842	1.710730	131.408528	1	1
2107	1	21.982942	1.748584	133.742943	1	1
2108	1	22.524036	1.752206	133.689352	1	1
2109	1	24.361936	1.739450	133.346641	1	1
2110	1	23.664709	1.738836	133.472641	1	1

2111 rows × 7 columns

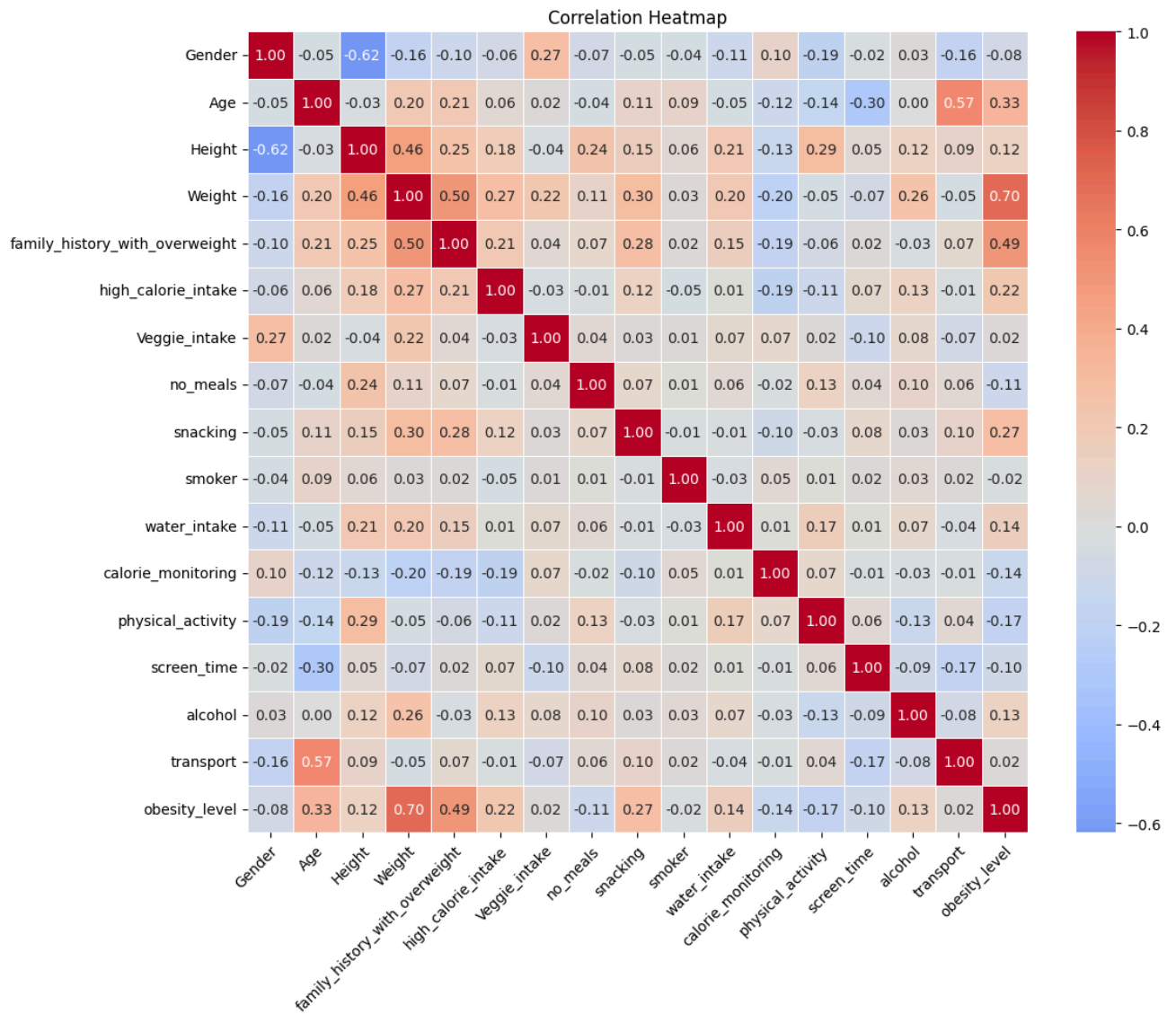
```
import seaborn as sns
import matplotlib.pyplot as plt

corr_matrix = lifesytle_cleaned.corr()

plt.figure(figsize=(12, 10))
heatmap = sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    center=0,
    fmt=".2f",
    linewidths=0.5
)

# Fix misaligned x-axis labels
heatmap.set_xticklabels(
    heatmap.get_xticklabels(),
    rotation=45,          # Rotate labels 45 degrees
    ha='right',           # Horizontal alignment to right
    rotation_mode='anchor' # Anchor rotation to label position
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
from imblearn.over_sampling import SMOTE # For handling class imbalance
import matplotlib.pyplot as plt

# 1. Load and prepare data
X = lifestyle_cleaned.drop(columns=['obesity_level'])
y = lifestyle_cleaned['obesity_level'].map({0: 'non-obesity', 1: 'obesity'}) # R

# 2. Convert categorical features (if any)
X = pd.get_dummies(X, drop_first=True)

# 3. Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 4. Scale features (important for Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Apply SMOTE to handle class imbalance
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

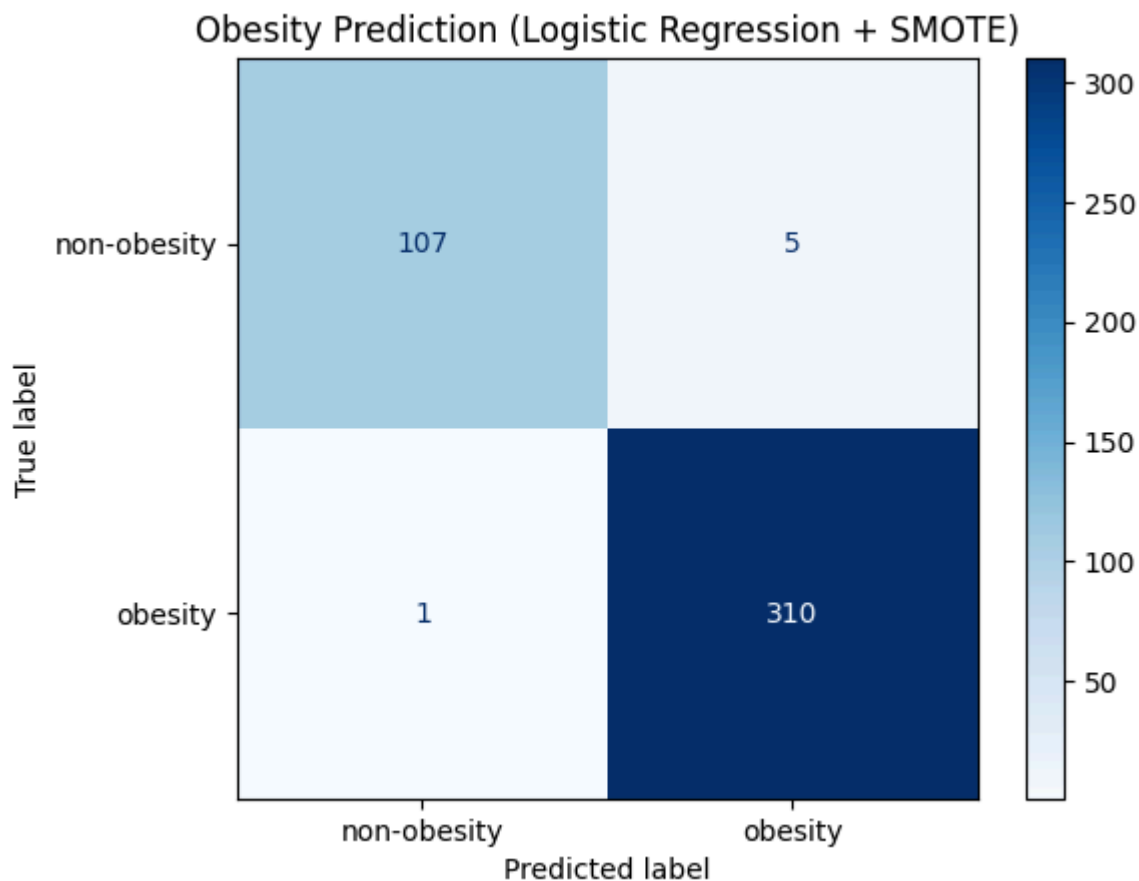
# 6. Train Logistic Regression model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_resampled, y_train_resampled) # Train on resampled data

# 7. Generate predictions
y_pred = model.predict(X_test_scaled)

# 8. Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=['non-obesity', 'obesity'])
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=['non-obesity', 'obesity']
)
disp.plot(cmap='Blues', values_format='d')
plt.title('Obesity Prediction (Logistic Regression + SMOTE)')
plt.show()

# 9. Classification Report
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=['non-obesity', 'obesity']))

# 10. (Optional) Compare class distribution before/after SMOTE
print("\nClass distribution BEFORE SMOTE:", pd.Series(y_train).value_counts())
print("Class distribution AFTER SMOTE:", pd.Series(y_train_resampled).value_counts())
```



Classification Report:				
	precision	recall	f1-score	support
non-obesity	0.99	0.96	0.97	112
obesity	0.98	1.00	0.99	311
accuracy			0.99	423
macro avg	0.99	0.98	0.98	423
weighted avg	0.99	0.99	0.99	423

```

Class distribution BEFORE SMOTE: obesity_level
obesity      1241
non-obesity   447
Name: count, dtype: int64
Class distribution AFTER SMOTE: obesity_level
obesity      1241
non-obesity   1241
Name: count, dtype: int64

```

```

import pandas as pd
import numpy as np
from scipy import stats # For calculating p-values

feature_names = X.columns.tolist() # Ensure this matches your actual features

# Calculate p-values for logistic regression coefficients
def get_logit_pvalues(model, X):
    """Calculate p-values for logistic regression coefficients."""
    p = model.predict_proba(X)[: , 1]
    logit = np.log(p / (1 - p))

```

```

    denom = 2 * (1 + np.cosh(logit))
    denom = np.tile(denom, (X.shape[1], 1)).T
    F_ij = np.dot((X / denom).T, X)
    cov = np.linalg.inv(F_ij)
    sigma = np.sqrt(np.diag(cov))
    z = model.coef_[0] / sigma
    return 2 * (1 - stats.norm.cdf(np.abs(z)))

p_values = get_logit_pvalues(model, X_train_resampled)

importance_df = pd.DataFrame({
    'Feature': ['const'] + feature_names,
    'Coefficient': np.concatenate([[model.intercept_[0]], model.coef_[0]]),
    'P_value': np.concatenate([[np.nan], p_values]) # No p-value for intercept
})

top_significant = importance_df[
    (importance_df['P_value'] < 0.05) &
    (importance_df['Feature'] != 'const')
].sort_values('P_value', ascending=True)

print("\nTop Statistically Significant Features (p < 0.05):")
print(top_significant[['Feature', 'Coefficient', 'P_value']].head(10))

plt.figure(figsize=(10, 6))
top_coefs = top_significant.sort_values('Coefficient', key=abs, ascending=False).
plt.barh(top_coefs['Feature'], top_coefs['Coefficient'], color=['red' if x < 0 else
plt.xlabel('Coefficient Value')
plt.title('Top 10 Features Influencing Obesity Prediction (p < 0.05)')
plt.axvline(0, color='black', linestyle='--')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

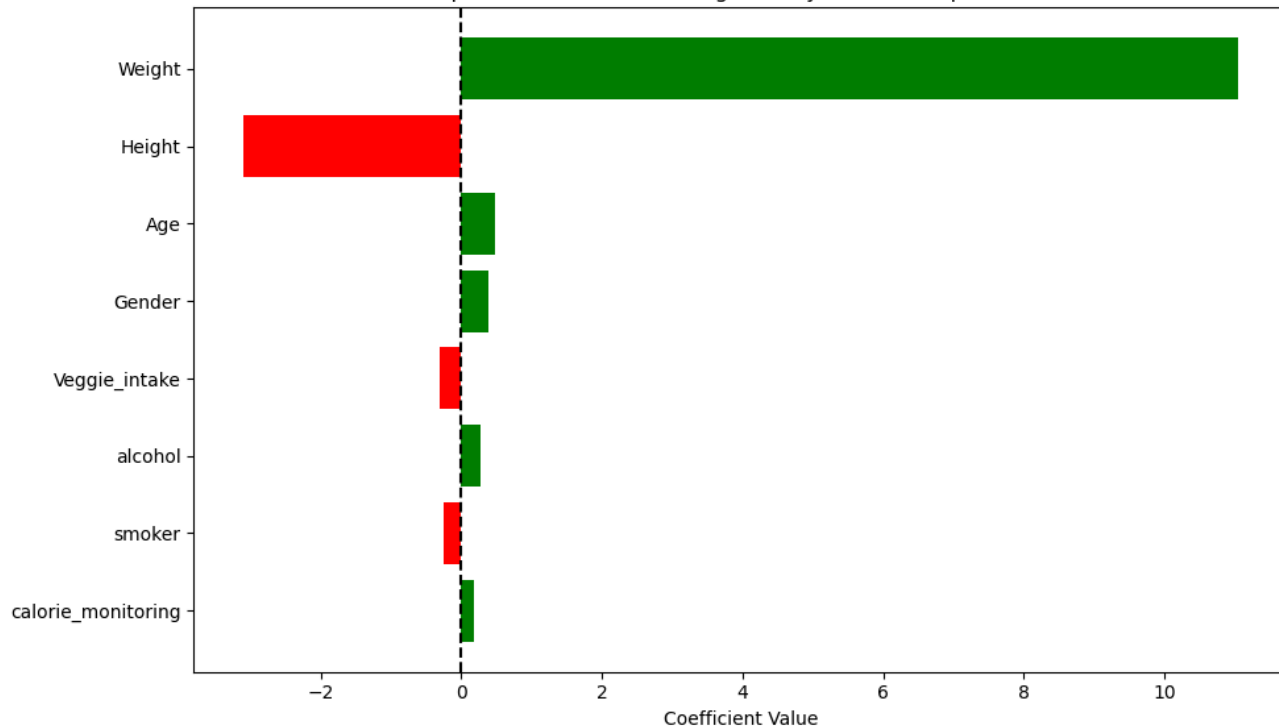
```



Top Statistically Significant Features ($p < 0.05$):

	Feature	Coefficient	P_value
4	Weight	11.051126	0.000000
3	Height	-3.096026	0.000000
2	Age	0.482613	0.009953
7	Veggie_intake	-0.301138	0.014397
12	calorie_monitoring	0.188327	0.017150
1	Gender	0.380434	0.022125
15	alcohol	0.277172	0.035138
10	smoker	-0.255644	0.041988

Top 10 Features Influencing Obesity Prediction ($p < 0.05$)



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create age_class column
lifestyle_cleaned['age_class'] = pd.cut(
    lifestyle_cleaned['Age'],
    bins=[0, 21, np.inf], # Corrected bin edges
    labels=['≤21', '>21'], # More descriptive labels
    right=False
)
```

```
# Verify the distribution
print("Age Class Distribution:")
print(lifesytle_cleaned['age_class'].value_counts(dropna=False))

# Visualize with improved styling
plt.figure(figsize=(8, 5))
sns.set_style("whitegrid")
ax = sns.countplot(data=lifesytle_cleaned, x='age_class', palette="viridis")

# Add annotations and styling
plt.title('Distribution of Age Classes', fontsize=14, pad=20)
plt.xlabel('Age Group', fontsize=12)
plt.ylabel('Count', fontsize=12)

# Add count labels on bars
for p in ax.patches:
    ax.annotate(f'{p.get_height():,0f}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center',
                xytext=(0, 5),
                textcoords='offset points')

plt.tight_layout()
plt.show()
```

```

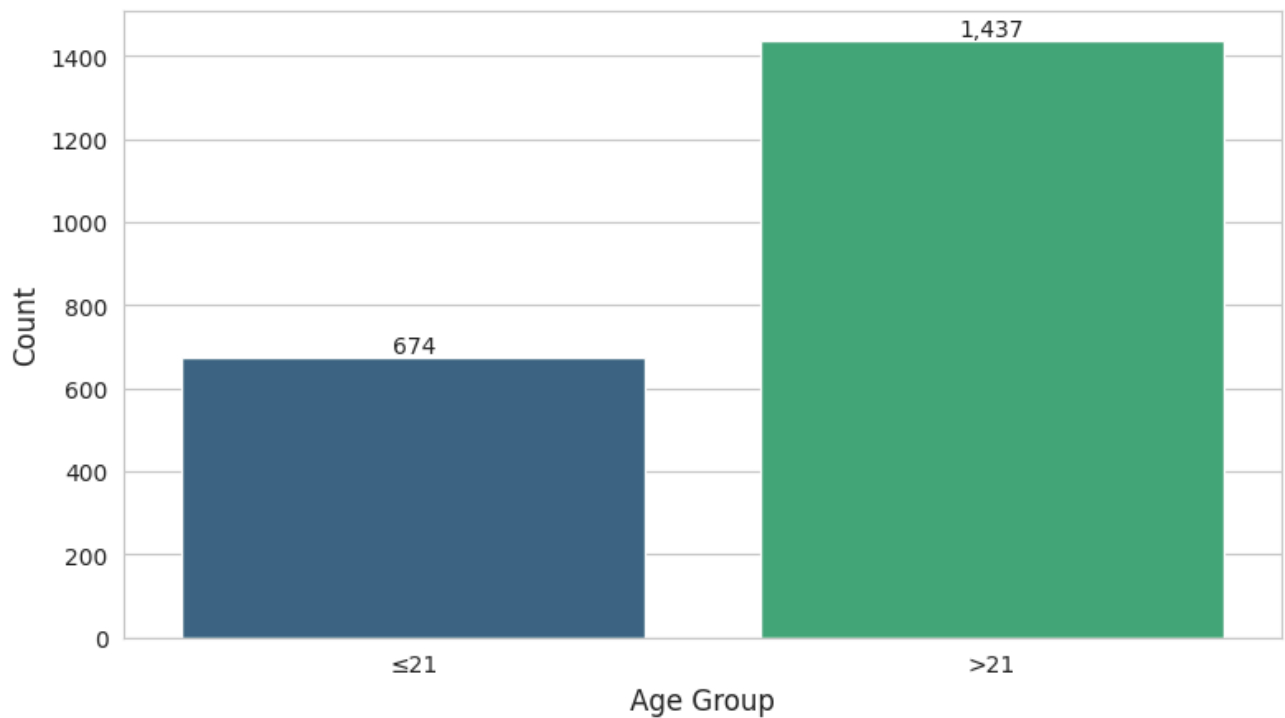
⇒ Age Class Distribution:
age_class
>21    1437
≤21     674
Name: count, dtype: int64
<ipython-input-147-6a73443b3ffc>:21: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in

```
ax = sns.countplot(data=lifesytle_cleaned, x='age_class', palette="viridis"
```

Distribution of Age Classes



```

import pandas as pd
import numpy as np

```

```

# Create the age_class column first (if not already done)
lifesytle_cleaned['age_class'] = pd.cut(
    lifesytle_cleaned['Age'],
    bins=[0, 21, np.inf],
    labels=['≤21', '>21'],
    right=False
)

```

```

# Create separate DataFrames
df_under21 = lifesytle_cleaned[lifesytle_cleaned['age_class'] == '≤21'].copy()
df_over21 = lifesytle_cleaned[lifesytle_cleaned['age_class'] == '>21'].copy()

```

```

# Verify the splits
print(f"≤21 DataFrame shape: {df_under21.shape}")
print(f">21 DataFrame shape: {df_over21.shape}")

```

```
# Optional: Verify age ranges
print("\nAge range verification:")
print("<=21 group age range:", df_under21['Age'].min(), "-", df_under21['Age'].max())
print(">21 group age range:", df_over21['Age'].min(), "-", df_over21['Age'].max())
```

```
# Optional: Save to separate files
df_under21.to_csv('under21_data.csv', index=False)
df_over21.to_csv('over21_data.csv', index=False)
```

```
↗ ≤21 DataFrame shape: (674, 19)
  >21 DataFrame shape: (1437, 19)
```

```
Age range verification:
≤21 group age range: 14.0 - 20.993067
>21 group age range: 21.0 - 61.0
```

```
adults_seniors_df = lifestyle_cleaned[
    lifestyle_cleaned['age_class'].isin(['Adults', 'Seniors'])
].copy()
```

```
adults_seniors_df
```

```
↗
```

	Gender	Age	Height	Weight	family_history_with_overweight	high_blood_pressure
0	1	21.000000	1.620000	64.000000		1
1	1	21.000000	1.520000	56.000000		1
2	0	23.000000	1.800000	77.000000		1
3	0	27.000000	1.800000	87.000000		0
4	0	22.000000	1.780000	89.800000		0
...	...	...	...	...		...
2105	1	21.285965	1.726920	131.335786		1
2107	1	21.982942	1.748584	133.742943		1

```
adults_df = lifestyle_cleaned[
    lifestyle_cleaned['age_class'].isin(['Adults'])
].copy()
```

```
adults_df
```



	Gender	Age	Height	Weight	family_history_with_overweight	high_blood_pressure
0	1	21.000000	1.620000	64.000000		1
1	1	21.000000	1.520000	56.000000		1
2	0	23.000000	1.800000	77.000000		1
3	0	27.000000	1.800000	87.000000		0
4	0	22.000000	1.780000	89.800000		0
...	...	...	...	...		...
2105	1	21.285965	1.726920	131.335786		1
2107	1	21.982942	1.748584	133.742943		1

```
seniors_df = lifesytle_cleaned[
    lifesytle_cleaned['age_class'].isin(['Seniors'])
].copy()

seniors_df
```



	Gender	Age	Height	Weight	family_history_with_overweight	high
21	1	52.000000	1.690000	87.000000		1
92	0	55.000000	1.780000	84.000000		1
133	1	61.000000	1.650000	66.000000		0
161	0	55.000000	1.650000	80.000000		0
169	1	45.000000	1.630000	77.000000		1
232	1	51.000000	1.590000	50.000000		1
252	0	56.000000	1.790000	90.000000		1
492	0	45.000000	1.700000	86.000000		0
1013	0	55.246250	1.769269	80.491339		0
1034	0	50.832559	1.745528	82.130728		1
1063	1	45.000000	1.675953	79.668320		1

```

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Prepare data (Adults & Seniors only)
X = seniors_df.drop(columns=['obesity_level', 'age_class', 'Gender', 'Age', 'Height'])
y = seniors_df['obesity_level'] # Target (must be binary: 0/1 or 'non-obesity')

# 2. One-hot encode categorical features (if any)
X = pd.get_dummies(X, drop_first=True)

# 3. Check for NaN values
if X.isnull().sum().any() or y.isnull().sum() > 0:
    print("Data contains NaN values. Please handle them before proceeding.")
else:
    # 4. Split the data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # 5. Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

```

```
X_test_scaled = scaler.transform(X_test)

# 6. Fit logistic regression with regularization
model = LogisticRegression(penalty='l2', solver='liblinear', max_iter=1000)
model.fit(X_train_scaled, y_train)

# 7. Extract coefficients and feature names
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

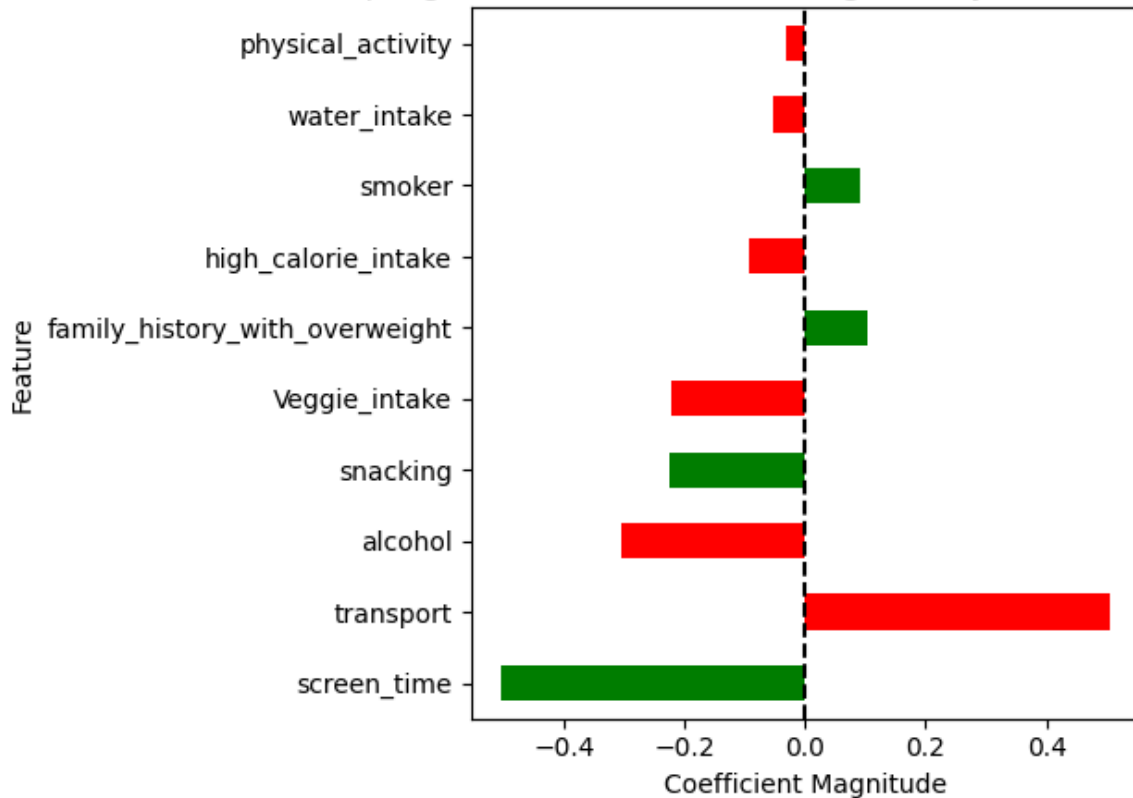
# 8. Print significant features
print(importance_df[importance_df['Coefficient'] != 0])

# 9. Visualize top features
plt.figure(figsize=(10, 6))
importance_df.sort_values('Coefficient', key=abs, ascending=False).head(10).p
    x='Feature', y='Coefficient',
    color=importance_df['Coefficient'].apply(lambda x: 'red' if x < 0 else 'g
    legend=False
)
plt.axvline(0, color='black', linestyle='--')
plt.xlabel('Coefficient Magnitude')
plt.title('Top Significant Features Predicting Obesity (Adults & Seniors)')
plt.tight_layout()
plt.show()
```

	Feature	Coefficient
0	family_history_with_overweight	0.102644
1	high_calorie_intake	-0.091446
2	Veggie_intake	-0.222243
3	no_meals	0.001597
4	snacking	-0.226140
5	smoker	0.091446
6	water_intake	-0.052872
8	physical_activity	-0.032905
9	screen_time	-0.503885
10	alcohol	-0.304607
11	transport	0.503885

<Figure size 1000x600 with 0 Axes>

Top Significant Features Predicting Obesity (Adults & Seniors)



```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Prepare data (Adults & Seniors only)
X = adults_df.drop(columns=['obesity_level', 'age_group', 'age_class', 'Gender'],
y = adults_df['obesity_level'] # Target (must be binary: 0/1 or 'non-obesity'/'o

# 2. One-hot encode categorical features (if any)
X = pd.get_dummies(X, drop_first=True)

# 3. Check for NaN values
if X.isnull().sum().any() or y.isnull().sum() > 0:
    print("Data contains NaN values. Please handle them before proceeding.")
```

else:

```
# 4. Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rand

# 5. Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Fit logistic regression with regularization
model = LogisticRegression(penalty='l2', solver='liblinear', max_iter=1000)
model.fit(X_train_scaled, y_train)

# 7. Extract coefficients and feature names
importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

# 8. Print significant features
print(importance_df[importance_df['Coefficient'] != 0])

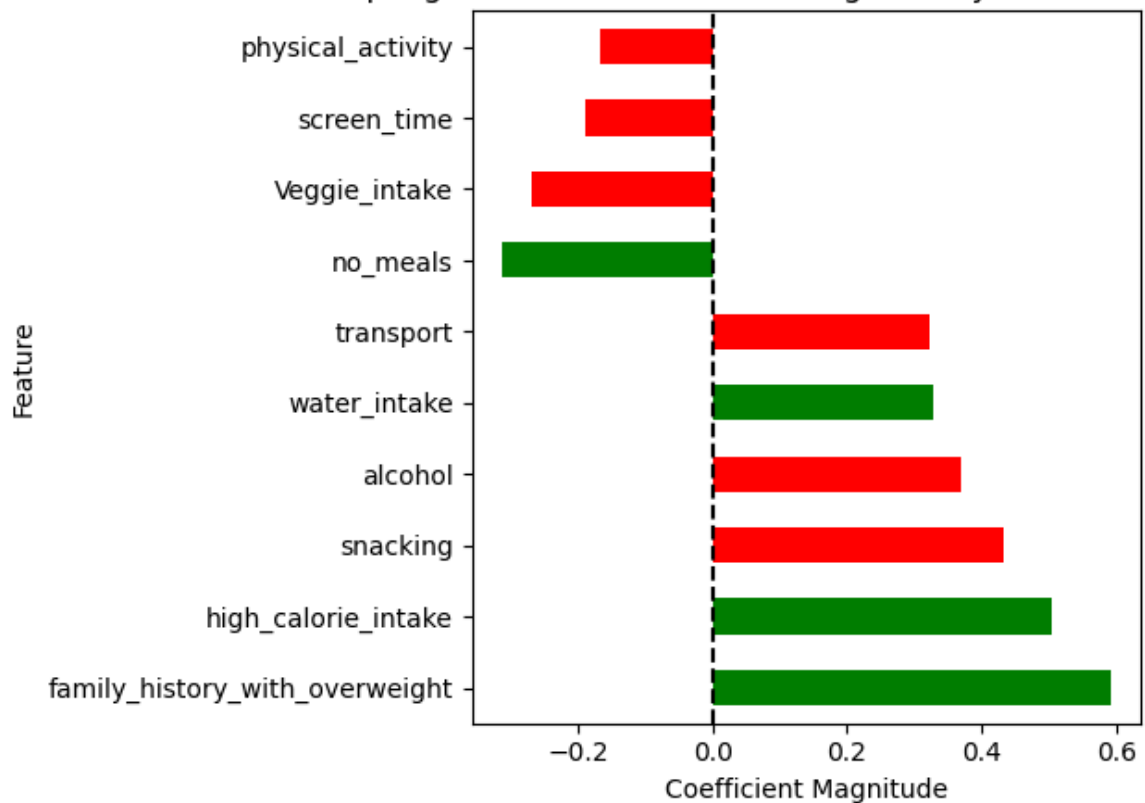
# 9. Visualize top features
plt.figure(figsize=(10, 6))
importance_df.sort_values('Coefficient', key=abs, ascending=False).head(10).p
    x='Feature', y='Coefficient',
    color=importance_df['Coefficient'].apply(lambda x: 'red' if x < 0 else 'g
    legend=False
)
plt.axvline(0, color='black', linestyle='--')
plt.xlabel('Coefficient Magnitude')
plt.title('Top Significant Features Predicting Obesity (Adults & Seniors)')
plt.tight_layout()
plt.show()
```



	Feature	Coefficient
0	family_history_with_overweight	0.591020
1	high_calorie_intake	0.502864
2	Veggie_intake	-0.267982
3	no_meals	-0.312708
4	snacking	0.433285
5	smoker	-0.093434
6	water_intake	0.327148
7	calorie_monitoring	-0.165883
8	physical_activity	-0.166990
9	screen_time	-0.188223
10	alcohol	0.368641
11	transport	0.321959

<Figure size 1000x600 with 0 Axes>

Top Significant Features Predicting Obesity (Adults & Seniors)





	Gender	Age	Height	Weight	family_history_with_overweight	high_blood_pressure
0	1	21.000000	1.620000	64.000000		1
1	1	21.000000	1.520000	56.000000		1
2	0	23.000000	1.800000	77.000000		1
3	0	27.000000	1.800000	87.000000		0
4	0	22.000000	1.780000	89.800000		0
...	...	...	...	...		...
2105	1	21.285965	1.726920	131.335786		1
2107	1	21.982942	1.748584	133.742943		1

```

import pandas as pd
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Prepare data (Adults & Seniors only)
X = adults_df.drop(columns=['obesity_level', 'age_class', 'age_group', 'Gender',
y = adults_df['obesity_level'] # Target (must be binary: 0/1 or 'non-obesity'/'o

# 2. One-hot encode categorical features (if any)
X = pd.get_dummies(X, drop_first=True)

# 3. Check for constant columns
constant_columns = [col for col in X.columns if X[col].nunique() <= 1]
if constant_columns:
    print(f"Constant columns identified and will be dropped: {constant_columns}")
    X = X.drop(columns=constant_columns)

# 4. Check for NaN values
if X.isnull().sum().any() or y.isnull().sum() > 0:
    print("Data contains NaN values. Please handle them before proceeding.")
else:
    # 5. Split the data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rand

    # 6. Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

```

```
# 7. Add intercept for statsmodels
X_train_scaled = sm.add_constant(X_train_scaled)

# 8. Fit logistic regression using statsmodels
try:
    model = sm.Logit(y_train, X_train_scaled).fit(dis=0) # dis=0 suppresses
except Exception as e:
    print(f"Error fitting model: {e}")
    raise

# 9. Extract coefficients and p-values
importance_df = pd.DataFrame({
    'Feature': ['Intercept'] + list(X.columns), # Include intercept in featu
    'Coefficient': model.params,
    'P-value': model.pvalues
})

# 10. Filter for significant features (p < 0.05)
significant_features = importance_df[importance_df['P-value'] < 0.05].sort_va

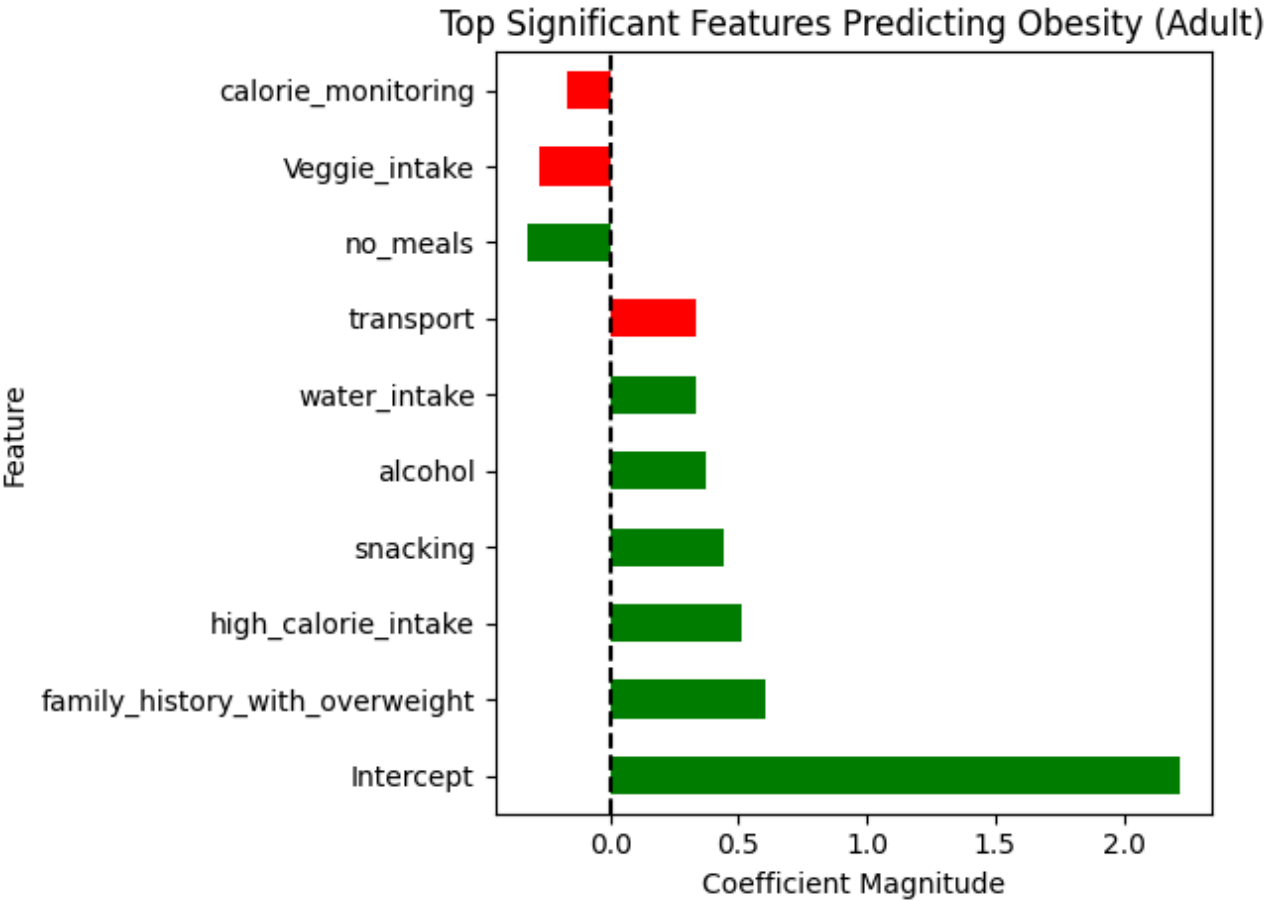
print("Statistically Significant Features (p < 0.05):")
print(significant_features)

# 11. Visualize significant features
if not significant_features.empty:
    plt.figure(figsize=(10, 6))
    significant_features.sort_values('Coefficient', key=abs, ascending=False)
    x='Feature', y='Coefficient',
    color=significant_features['Coefficient'].apply(lambda x: 'red' if x
    legend=False
    )
    plt.axvline(0, color='black', linestyle='--')
    plt.xlabel('Coefficient Magnitude')
    plt.title('Top Significant Features Predicting Obesity (Adult)')
    plt.tight_layout()
    plt.show()
else:
    print("No significant features (p < 0.05) found.")
```

Statistically Significant Features (p < 0.05):

	Feature	Coefficient	P-value
const	Intercept	2.215343	2.859245e-79
x1	family_history_with_overweight	0.601447	4.349389e-13
x2	high_calorie_intake	0.509952	1.466499e-10
x5	snacking	0.440973	1.115767e-07
x11	alcohol	0.377429	9.190203e-05
x7	water_intake	0.335495	4.747103e-04
x4	no_meals	-0.321530	1.367231e-03
x12	transport	0.333314	3.129684e-03
x3	Veggie_intake	-0.275743	7.724857e-03
x8	calorie_monitoring	-0.167871	2.555420e-02

<Figure size 1000x600 with 0 Axes>



adults_df



	Gender	Age	Height	Weight	family_history_with_overweight	high_blood_pressure
0	1	21.000000	1.620000	64.000000		1
1	1	21.000000	1.520000	56.000000		1

```

import pandas as pd
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Prepare data (Adults & Seniors only)
X = df_under21.drop(columns=['obesity_level', 'age_class', 'age_group', 'Gender',
y = df_under21['obesity_level'] # Target (must be binary: 0/1 or 'non-obesity')

# 2. One-hot encode categorical features (if any)
X = pd.get_dummies(X, drop_first=True)

# 3. Check for constant columns
constant_columns = [col for col in X.columns if X[col].nunique() <= 1]
if constant_columns:
    print(f"Constant columns identified and will be dropped: {constant_columns}")
    X = X.drop(columns=constant_columns)

# 4. Check for NaN values
if X.isnull().sum().any() or y.isnull().sum() > 0:
    print("Data contains NaN values. Please handle them before proceeding.")
else:
    # 5. Split the data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rand

    # 6. Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # 7. Add intercept for statsmodels
    X_train_scaled = sm.add_constant(X_train_scaled)

    # 8. Fit logistic regression using statsmodels
    try:
        model = sm.Logit(y_train, X_train_scaled).fit(dis=0) # dis=0 suppresses
except Exception as e:

```